

Name : Sanika Mangrulkar

PRN : 202501050015

Practical 4

1)

Program :

```
import pandas as pd

# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
series = pd.Series(numbers)

# Grouping by even and odd numbers and calculating the mean
grouped = series.groupby(series % 2 == 0).mean()

# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]

print("Mean of even and odd numbers:")

print(grouped)
```

Output :

Average time

0.081 s

80.50 ms

Maximum time

0.218 s

218.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1218 ms

Debug

Expected output

Actual output

1 2 3 4 5 6 7 8 9 10

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers:

Mean of even and odd numbers:

Odd ... 5.0

Odd ... 5.0

Even ... 6.0

Even ... 6.0

dtype: float64

dtype: float64

Test case 255 ms

Debug

Expected output

Actual output

4 4 4 6 6 2 2 2 10 12 14 16

4 4 4 6 6 2 2 2 10 12 14 16

Mean of even and odd numbers:

Mean of even and odd numbers:

Even ... 6.833333

Even ... 6.833333

dtype: float64

dtype: float64

Test case 368 ms

Debug

Expected output

Actual output

1 5 7 9 11 13 15 17 111 21 25

1 5 7 9 11 13 15 17 111 21 25

Mean of even and odd numbers:

Mean of even and odd numbers:

Odd ... 21.363636

Odd ... 21.363636

2)

Program :

```
import pandas as pd
```

```
# Provided dictionary of lists
```

```
data = {
```

```
    'Name': ['Alice', 'Bob', 'Charlie'],
```

```
    'Age': [25, 30, 35],
```

```
}
```

```
# Convert the dictionary to a DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
# Adding a new row
```

```
new_name=input("New name: ")
```

```
new_age=int(input("New age: "))
```

```
new_row={'Name':new_name,'Age':new_age}
```

```
df=df.append(new_row,ignore_index=True)
```

```
# Display the DataFrame after adding a new row
```

```
print("After adding a row:\n",df)
```

```
# Modifying a row
```

```
modify_row_index=int(input("Index of row to modify: "))
```

```
new_ag_value=int(input("New age: "))
```

```
df.at[modify_row_index,'Age']=new_ag_value
```

```
# Display the DataFrame after modifying a row
```

```
print("After modifying a row:")
```

```
print(df)
```

```
# Deleting a row
```

```
delete_row_index=int(input("Index of row to delete: "))  
df=df.drop(delete_row_index).reset_index(drop=True)  
# Display the DataFrame after deleting a row  
print("After deleting a row:")  
print(df)
```

```
# Adding a new column  
genders=input("Enter genders separated by space: ").split()  
df['Gender']=genders
```

```
# Display the DataFrame after adding a new column  
print("After adding a new column:")  
print(df)
```

```
# Modifying a column  
df['Name']=df['Name'].str.upper()  
# Display the DataFrame after modifying a column  
print("After modifying a column:")  
print(df)
```

```
# Deleting a column  
df=df.drop(columns=['Age'])  
# Display the DataFrame after deleting a column  
print("After deleting a column:")  
print(df)
```

Output :

Average time: **1.185 s** (1185.50 ms) | Maximum time: **1.216 s** (1216.00 ms) | **1 out of 1 shown test case(s) passed** | **1 out of 1 hidden test case(s) passed**

Test case 1: **1216 ms** [Debug]

Expected output	Actual output
Original DataFrame:	Original DataFrame:
.....Name..Age	Name..Age
0....Alice...25	0....Alice...25
1....Bob...30	1....Bob...30
2..Charlie...35	2..Charlie...35
New name: Susan	New name: Susan
New age: 29	New age: 29
After adding a row:	After adding a row:
.....Name..Age	Name..Age
0....Alice...25	0....Alice...25
1....Bob...30	1....Bob...30
2..Charlie...35	2..Charlie...35
3....Susan...29	3....Susan...29
Index of row to modify: 0	Index of row to modify: 0
New age: 27	New age: 27
After modifying a row:	After modifying a row:
.....Name..Age	Name..Age
0....Alice...27	0....Alice...27

3)

Program :

```
import pandas as pd
```

```
# Read the text file into a DataFrame
```

```
file = input()
```

```
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
```

```
# Display the first five rows
```

```
print("First five rows:")
```

```
print(data.head())
```

```
# Calculate the average age of students (limited to 2 decimal places)
```

```
average_age = round(data["Age"].mean(), 2)
```

```
print(f"Average age: {average_age}")
```

```
# Filter students with a grade up to 'B'
```

```
filtered_students = data[data["Grade"] <= "B"]
```

```
print("Students with a grade up to B")
```

```
print(filtered_students)
```

Output :

3

Read the text file into a DataFrame

Average time
0.172 s
172.50 ms

Maximum time
0.236 s
236.00 ms

✓ 1 out of 1 shown test case(s) passed
✓ 1 out of 1 hidden test case(s) passed

Test case 1 236 ms Debug

Expected output

Actual output

studentdata.txt	studentdata.txt
First five rows:	First five rows:
...Name Age Grade	...Name Age Grade
0 John 25 A	0 John 25 A
1 Alin 22 B	1 Alin 22 B
2 Emma 24 A	2 Emma 24 A
3 Mike 23 C	3 Mike 23 C
4 Sony 21 B	4 Sony 21 B
Average age: 23.29	Average age: 23.29
Students with a grade up to B	Students with a grade up to B
...Name Age Grade	...Name Age Grade
0 John 25 A	0 John 25 A
1 Alin 22 B	1 Alin 22 B
2 Emma 24 A	2 Emma 24 A
4 Sony 21 B	4 Sony 21 B
5 Amia 26 A	5 Amia 26 A

4)

Program :

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
# Convert the Date column to datetime format
```

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
# Add a Year-Month column by formatting the Date column
```

```
df['YearMonth'] = df['Date'].dt.to_period('M')
```

```
# Calculate total sales for each row
```

```
df['Total_Sales'] = df['Quantity'] * df['Price']
```

```
# Group the data by Year-Month and calculate total sales for each month
```

```
monthly_sales = df.groupby('YearMonth')['Total_Sales'].sum()
```

```
# Find the month with the highest total sales
```

```
best_month = monthly_sales.idxmax()
```

```
highest_sales = monthly_sales.max()
```

```
print(f"Best month: {best_month}")  
  
print(f"Total sales: ${highest_sales:.2f}")
```

Output :

The screenshot shows a code execution interface. At the top, a code editor displays a comment `# Load the data`. Below the editor, performance metrics are shown: Average time is 0.110 s (109.67 ms) and Maximum time is 0.223 s (223.00 ms). Test results indicate that 1 out of 1 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. A 'Test case 1' summary shows a duration of 223 ms and a 'Debug' button. Below this, a table compares 'Expected output' and 'Actual output'. Both show the file 'sales_data.csv' and the same printed output: 'Best month: 2025-01' and 'Total sales: \$1210.00'.

Performance Metrics		Test Results	
Average time	0.110 s 109.67 ms	Maximum time	0.223 s 223.00 ms
		1 out of 1 shown test case(s) passed	
		2 out of 2 hidden test case(s) passed	

Test case 1 (223 ms)	
Expected output	Actual output
sales_data.csv	sales_data.csv
Best month: 2025-01	Best month: 2025-01
Total sales: \$1210.00	Total sales: \$1210.00

5)

Program :

```
import pandas as pd  
  
# Prompt the user for the file name  
file_name = input()  
  
# Load the data  
df = pd.read_csv(file_name)  
  
product_sales = df.groupby('Product')['Quantity'].sum()
```



```
# Find the product with the highest total quantity sold
```

```
best_product = product_sales.idxmax()
```

```
highest_quantity = product_sales.max()
```

```
# Display the result
```

```
print(f"Best selling product: {best_product}")
```

```
print(f"Total quantity sold: {highest_quantity}")
```

Output :

The screenshot displays a test runner interface with the following components:

- Performance Metrics:**
 - Average time: 0.073 s (73.00 ms)
 - Maximum time: 0.154 s (154.00 ms)
- Test Results Summary:**
 - 1 out of 1 shown test case(s) passed
 - 2 out of 2 hidden test case(s) passed
- Test Case Details:**
 - Test case 1: 154 ms, with buttons for Debug, List, and Run.
 - Expected output:**
 - sales_data.csv
 - Best selling product: Product A
 - Total quantity sold: 23
 - Actual output:**
 - sales_data.csv
 - Best selling product: Product A
 - Total quantity sold: 23

6)

Program :

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```

file_name = input()

# Load the data

df = pd.read_csv(file_name)

city_sales = df.groupby('City')['Quantity'].sum()

# Find the city that sold the most products

best_city = city_sales.idxmax()

highest_quantity = city_sales.max()

# write the code..

# Display the result

print(f"City sold the most products: {best_city}")

```

Output :

Average time 0.084 s 84.33 ms	Maximum time 0.177 s 177.00 ms	✓ 1 out of 1 shown test case(s) passed ✓ 2 out of 2 hidden test case(s) passed
--	---	---

Test case 1 **177 ms**
Debug

Expected output	Actual output
sales_data.csv	sales_data.csv
City sold the most products: Los Angeles	City sold the most products: Los Angeles

7)

Program :

```
import pandas as pd

from itertools import combinations

from collections import Counter


# Prompt user to input the file name

file_name = input()


# Read data from the specified CSV file

df = pd.read_csv(file_name)


count_pairs=Counter()


# write the code


for date,group in df.groupby("Date"):

    products = group['Product']

    for pair in combinations(sorted(products), 2):

        count_pairs[pair] +=1


# Output the most frequent product pairs


best_count= max(count_pairs.values())

best_pair= [pair for pair,val in count_pairs.items() if val==best_count ]

for pair in best_pair:

    print(f"{pair[0]} and {pair[1]}: {best_count} times")
```

Output :

The screenshot displays a test runner interface. At the top, it shows performance metrics: Average time (0.094 s, 93.67 ms) and Maximum time (0.203 s, 203.00 ms). To the right, it indicates that 1 out of 1 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. Below this, a section for 'Test case 1' (203 ms) includes a 'Debug' button and icons for a list and a document. The main part of the interface is a table comparing 'Expected output' and 'Actual output'. Both columns show the file 'sales_data.csv' and two lines of text: 'Product · A · and · Product · B : · 2 · times' and 'Product · A · and · Product · C : · 2 · times'.

Expected output	Actual output
sales_data.csv	sales_data.csv
Product · A · and · Product · B : · 2 · times	Product · A · and · Product · B : · 2 · times
Product · A · and · Product · C : · 2 · times	Product · A · and · Product · C : · 2 · times

8)

Program :

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
# 1. Display the first 5 rows of the dataset
```

```
print(data.head(5))
```

```
# 2. Display the last 5 rows of the dataset
```

```
print(data.tail(5))
```

```
# 3. Get the shape of the dataset
```

```
print(data.shape)
```

4. Get a summary of the dataset (info)

```
print(data.info())
```

5. Get basic statistics of the dataset

```
print(data.describe())
```

6. Check for missing values

```
print(data.isnull().sum())
```

7. Fill missing values in the 'Age' column with the median age

8. Fill missing values in the 'Embarked' column with the mode

9. Drop the 'Cabin' column due to many missing values

10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'

Output :

Average time

0.937 s

937.00 ms

Maximum time

0.937 s

937.00 ms

1 out of 1 shown test case(s) passed

Test case 1

937 ms

Debug

Expected output

Actual output

PassengerId Survived Pclass Fare Cabin Embarked

0 1 0 3 7.2500 NaN S

1 2 1 1 71.2833 C85

2 3 1 3 7.9250 NaN S

3 4 1 1 53.1000 C123

4 5 0 3 8.0500 NaN S

[5 rows x 12 columns]

PassengerId Survived Pclass Fare Cabin Embarked

886 887 0 2 13.00 NaN S

887 888 1 1 30.00 B42 S

888 889 0 3 23.45 NaN S

889 890 1 1 30.00 C148

PassengerId Survived Pclass Fare Cabin Embarked

0 1 0 3 7.2500 NaN S

1 2 1 1 71.2833 C85

2 3 1 3 7.9250 NaN S

3 4 1 1 53.1000 C123

4 5 0 3 8.0500 NaN S

[5 rows x 12 columns]

PassengerId Survived Pclass Fare Cabin Embarked

886 887 0 2 13.00 NaN S

887 888 1 1 30.00 B42 S

888 889 0 3 23.45 NaN S

889 890 1 1 30.00 C148

9)

Program :

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
df = pd.DataFrame(data)
```

```
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
```

```
df['FamilySize'] = df['SibSp'] + df['Parch']
```

```
df['IsAlone'] = (df['FamilySize'] == 0).astype(int)
```

```
# 2. Convert 'Sex' to numeric (male: 0, female: 1)
```

```
df['Sex'] = df['Sex'].map({'male' : 0, 'female' : 1})
```

```
# 3. One-hot encode the 'Embarked' column
```

```
df = pd.get_dummies(df, columns=['Embarked'])
```

```
# 4. Get the mean age of passengers
```

```
mean_age = df['Age'].mean()
```

```
# 5. Get the median fare of passengers
```

```
median_fare = df['Fare'].median()
```

```
# 6. Get the number of passengers by class
```

```
passengers_by_class = df['Pclass'].value_counts()
```

```
# 7. Get the number of passengers by gender
```

```
passengers_by_gender = df['Sex'].value_counts()
```

```
# 8. Get the number of passengers by survival status
```

```
passengers_by_survival = df['Survived'].value_counts()
```

```
# 9. Calculate the survival rate
```

```
survival_rate = df['Survived'].mean()
```

```
# 10. Calculate the survival rate by gender
```

```
survival_rate_by_gender = df.groupby('Sex')['Survived'].mean()
```

```
print(mean_age)
```

```
print(median_fare)
```

```
print(passengers_by_class)
```

```
print(passengers_by_gender)
```

```
print(passengers_by_survival)
```

```
print(survival_rate)
```

```
print(survival_rate_by_gender)
```

Output :

0.878 s 878.00 ms	0.878 s 878.00 ms	1 out of 1 shown test case(s) passed
Test case 1 878 ms Debug		
Expected output	Actual output	
29.69911764705882	29.69911764705882	
14.4542	14.4542	
3...491	3...491	
1...216	1...216	
2...184	2...184	
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64	
0...577	0...577	
1...314	1...314	
Name: Sex, dtype: int64	Name: Sex, dtype: int64	
0...549	0...549	
1...342	1...342	
Name: Survived, dtype: int64	Name: Survived, dtype: int64	
0.3838383838383838	0.3838383838383838	

10)

Program :

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
df = pd.DataFrame(data)
```

```
# 1. Calculate the survival rate by class
```

```
grouped = df.groupby('Pclass')['Survived'].mean()
```

```
print(grouped)
```

```
# 2. Calculate the survival rate by embarked location
```

```
print(df.groupby('Embarked_S')['Survived'].mean())
```


3. Calculate the survival rate by family size

```
print(df.groupby('FamilySize')['Survived'].mean())
```

4. Calculate the survival rate by being alone

```
print(data.groupby('IsAlone')['Survived'].mean())
```

5. Get the average fare by class

```
print(data.groupby('Pclass')['Fare'].mean())
```

6. Get the average age by class

```
print(data.groupby('Pclass')['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

```
print(data[data['Survived']==1]['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

```
print(data[data['Survived']==0]['Pclass'].value_counts())
```

Output :

1

import pandas as pd

Average time
0.394 s
394.00 ms

Maximum time
0.394 s
394.00 ms

✓ 1 out of 1 shown test case(s) passed

✓ Test case 1 394 ms

Debug

Expected output	Actual output
Pclass	Pclass
1...0.629630	1...0.629630
2...0.472826	2...0.472826
3...0.242363	3...0.242363
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Embarked_S	Embarked_S
0...0.506073	0...0.506073
1...0.336957	1...0.336957
Name: Survived, dtype: float64	Name: Survived, dtype: float64
FamilySize	FamilySize
0...0.303538	0...0.303538
1...0.552795	1...0.552795
2...0.578431	2...0.578431
3...0.724138	3...0.724138
4...0.200000	4...0.200000
5...0.136364	5...0.136364
6...0.333333	6...0.333333
7...0.000000	7...0.000000

11)

Program :

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
df = pd.DataFrame(data)
```

```
# 1. Get the number of survivors by gender
```

```
print(df[df['Survived']==1]['Sex'].value_counts())
```

```
# print((df.groupby('Sex')['Survived']==1).value_counts())
```

2. Get the number of non-survivors by gender

```
print(data[data['Survived']==0]['Sex'].value_counts())
```

3. Get the number of survivors by embarked location

```
print(data[data['Survived']==1]['Embarked_S'].value_counts())
```

4. Get the number of non-survivors by embarked location

```
print(data[data['Survived']==0]['Embarked_S'].value_counts())
```

5. Calculate the percentage of children (Age < 18) who survived

```
print(data[data['Age']<18]['Survived'].mean())
```

6. Calculate the percentage of adults (Age >= 18) who survived

```
print(data[data['Age']>17]['Survived'].mean())
```

7. Get the median age of survivors

```
print(data[data['Survived']==1]['Age'].median())
```

8. Get the median age of non-survivors

```
print(data[data['Survived']==0]['Age'].median())
```

9. Get the median fare of survivors

```
print(data[data['Survived']==1]['Fare'].median())
```

10. Get the median fare of non-survivors

```
print(data[data['Survived']==0]['Fare'].median())
```

Output :

Average time
0.383 s
383.00 ms

Maximum time
0.383 s
383.00 ms

✓ 1 out of 1 shown test case(s) passed

Test case 1 383 ms Debug

Expected output

Actual output

female....233	female....233
male.....109	male.....109
Name:Sex,-dtype:-int64	Name:Sex,-dtype:-int64
male.....468	male.....468
female.....81	female.....81
Name:Sex,-dtype:-int64	Name:Sex,-dtype:-int64
1....217	1....217
0....125	0....125
Name:Embarked_S,-dtype:-int64	Name:Embarked_S,-dtype:-int64
1....427	1....427
0....122	0....122
Name:Embarked_S,-dtype:-int64	Name:Embarked_S,-dtype:-int64
0.5398230088495575	0.5398230088495575
0.3810316139767055	0.3810316139767055
28.0	28.0
28.0	28.0
26.0	26.0
10.5	10.5